

Stammtisch — an AI for Schieber Jass

Progress log

Dario Hug

last update: 2026-09-26

At a glance

Goal	A Schieber agent that beats strong human players in duplicate matches.
State of the art	Strong-amateur level; no superhuman result published (HSLU, Uni Bern, ETH).
Data	1'821'878 human Swisslos games (2017-10-08 – 2018-04-15), 4'608 players, 65.6 M card plays.
Engine	C++ bitboards, 8.9 M rounds/s on a ThinkPad (14 threads), replays all human games.
Current phase	■ P0 done ■ P1 next

1 Plan

The approach follows what worked for imperfect-information games at scale (Pluribus/ReBeL for poker, DeepNash for Stratego, NooK for Bridge): imitate humans first, then improve by self-play, and add search over hidden cards at play time.



	Work	Exit criterion	
P0	Rules engine, simulator, monitoring, data ingest, validation	All human games replay; >1 M rounds/s	■
P1	Duplicate-deal arena + Elo; Weis/Stöck and multipliers; train/val/test split by date and player; jass-server client	Arena reports significant differences at ± 1 point/round	
P2	Determinized search (PIMC): sample hidden hands, solve each deal exactly with $\alpha\beta$	Beats heuristic baseline clearly	
P3	Transformer policy on human games (card play + trump/push head)	Top-1 human-move accuracy; beats P2 without search	
P4	Network predicting who holds which card from the public history	Lower hand-prediction loss than uniform sampling; improves P2	
P5	Self-play (PPO / R-NaD), KL-regularized to the human policy, league of past versions	Beats P3 and P4; stays playable with human partners	
P6	Test-time search with learned policy/value (IS-MCTS, ReBeL-style)	Best agent in arena	
P7	Web UI, duplicate matches vs. strong players	Positive score over ≥ 1000 duplicate deals	

Compute. The ThinkPad (14 threads, 30 GB RAM, no CUDA GPU) covers P0–P4 with small networks. P5/P6 at full scale will likely need a rented GPU; the C++ **VecEnv** keeps simulation off the critical path.

2 Rules and encoding

Encoding matches HSLU **jass-kit** so data and agents are interchangeable. Seats play counter-clockwise ($\text{next}(p) = p + 3 \bmod 4$), teams are $\{0, 2\}$ and $\{1, 3\}$. A hand is a 36-bit set in one `uint64`.

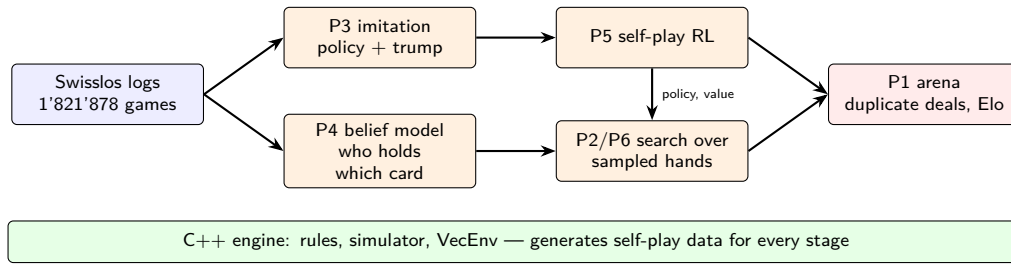


Figure 1: How the parts fit together. The engine produces unlimited self-play data; human logs bootstrap the policy and the belief model.

	A	K	Q	J	10	9	8	7	6
D Schellen	0	1	2	3	4	5	6	7	8
H Rosen	9	10	11	12	13	14	15	16	17
S Schilten	18	19	20	21	22	23	24	25	26
C Eichel	27	28	29	30	31	32	33	34	35

Trump order: J > 9 > A > K > Q > 10 > 8 > 7 > 6
Points: trump J 20, 9 14; A 11, 10 10, K 4, Q 3, J 2; Obenabe/Undeufe: 8 = 8, Undeufe 6 = 11 / A = 0. 152 + 5 (last trick) = 157; all tricks = 257.

Implemented: follow suit; trumping in is always allowed; no under-trumping unless only trumps are left; the Puur (trump J) never has to be played; push to partner. **Not yet:** Weis, Stöck, trump multipliers (P1). The Swisslos logs contain none of these either.

3 Data

The Swisslos logs (collected by HSLU for DL4G) contain every trick of each game, so all four hands can be reconstructed exactly. There are 65.6 M labelled card decisions. A separate file holds 359 825 trump decisions with the hand.

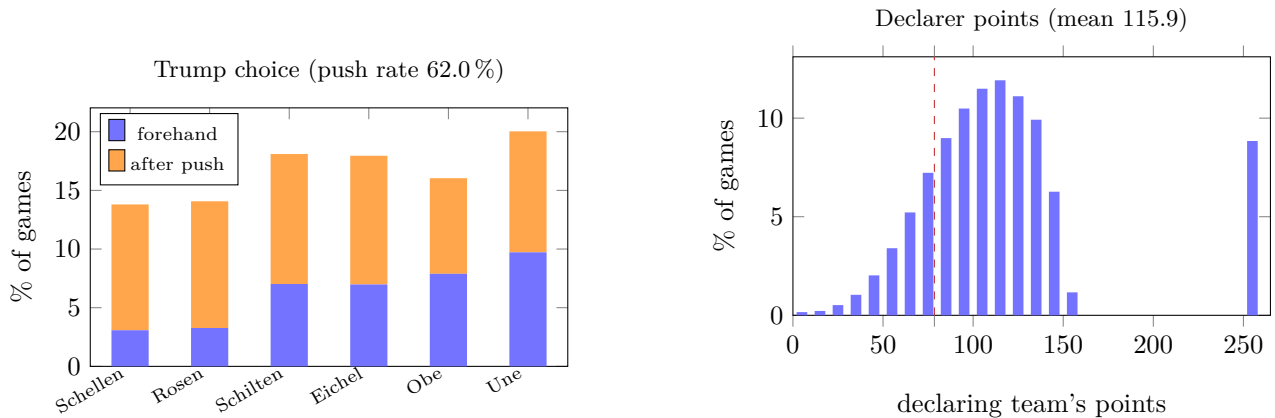
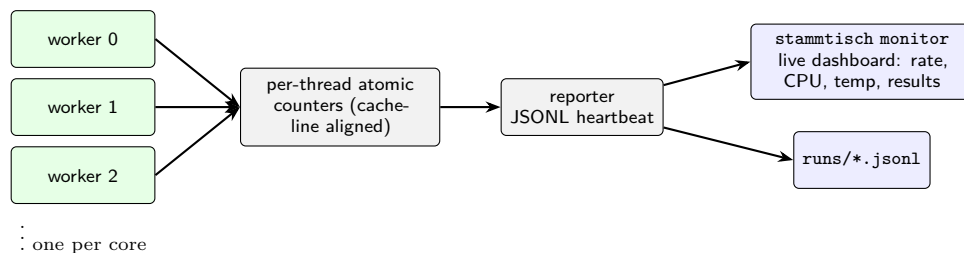
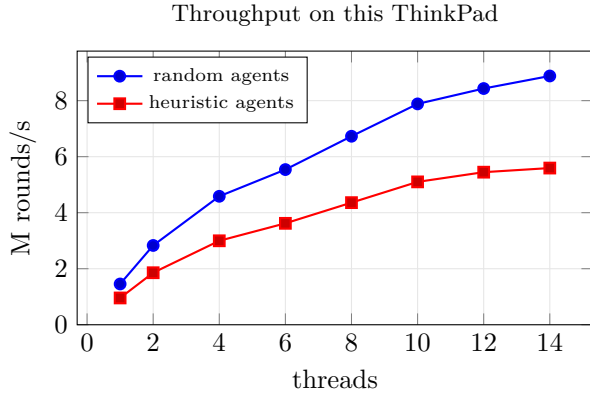


Figure 2: Left: forehand often pushes (62.0%); black suits and Undeufe are declared directly most often. Right: the declaring team wins 81.0 % of rounds (dashed: half of 157); 9.0 % of games are matches (257).

4 Simulator





- Bitboard hands; legal moves in a few bit operations.
- 956k rounds/s on one thread, 5.6M with all 14 (heuristic agents). Only 2 of the cores are performance cores, so scaling is sub-linear.
- Laptop numbers depend on turbo and power limits (best of 3 shown). Long runs at $\approx 100^\circ\text{C}$ vary by up to $\pm 25\%$. The monitor shows frequency and temperature next to the rate.
- Duplicate deals: each deal is played twice with seats swapped, which cancels most card luck.
- VecEnv C API: N rounds stepped in one call, with numpy views for learning agents.

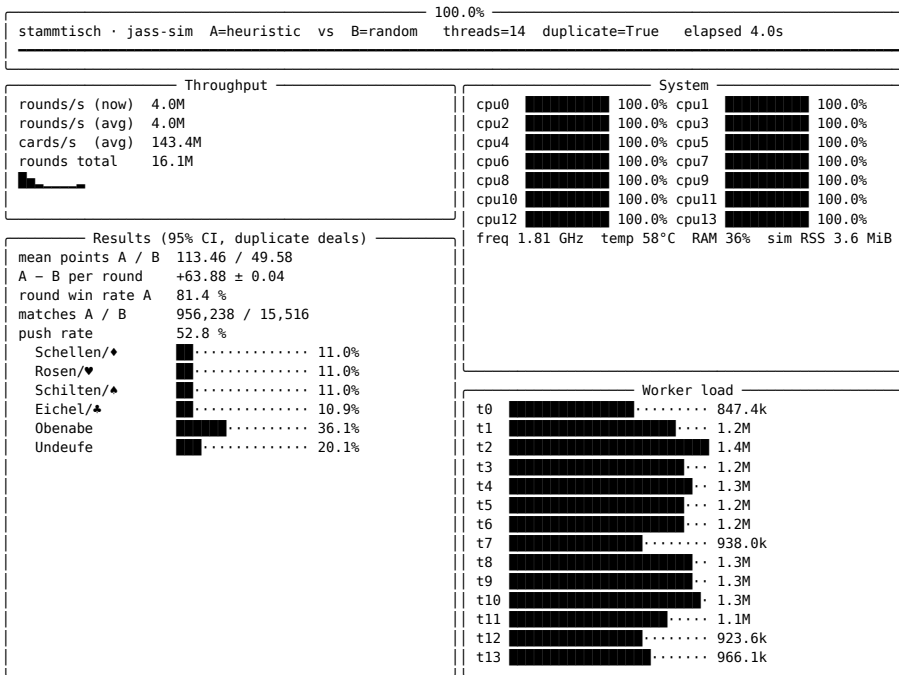


Figure 3: `make monitor`: live dashboard (final frame of a 4s run; heuristic vs. random, 14 threads). Every heartbeat is also logged to `runs/*.jsonl`.

Validation

Check	Result
Swisslos games replayed (every card legal, winners and points exact)	1'821'878 / 1'821'878
Legal moves vs. HSLU <code>jass-kit</code> , 119073 random states	173 differ, all <code>jass-kit</code> bug*
Trick winners vs. <code>jass-kit</code> , 20000 random tricks	identical
Heuristic vs. random (2M duplicate deals, points/round, 95 % CI)	+63.9 $\pm$ 0.1, win rate 81.4 %
Random vs. random / heuristic vs. heuristic	≈ 0 (sanity)

* With two trumps already in a trick, `jass-kit` takes the one with the larger card index as highest instead of the higher trump rank, and then allows under-trumping. None of the human games hit this case (the Swisslos server enforces the rule), so it only affects bots trained with `jass-kit`.

5 Log

2026-09-26 — P0 complete

Literature and project survey. Found the Swisslos logs mirrored in public DL4G student repositories;

drafted a request to HSLU (Prof. Th. Koller) for permission to use them. Built the C++ engine, multi-threaded simulator with live monitoring, and `VecEnv`. Validated against all human games and `jass-kit`; found one `jass-kit` rule bug.

Next — P1

Implement Weis/Stöck and multipliers (with rule variants), arena with Elo and significance tests, data splits, jass-server client for the HSLU tournament.

References

- HSLU DL4G, `jass-kit-py`: <https://github.com/thomas-koller/jass-kit-py>
- Niklaus et al., *Survey of AI for Card Games and its Application to the Swiss Game Jass*, SDS 2019. arXiv:1906.04439
- Uni Bern theses: AlphaZero for Jass (Zenodo 6400547), Approximate Nash Equilibria for Jass (Zenodo 7733443)
- Brown & Sandholm, *Superhuman AI for multiplayer poker* (Pluribus), Science 2019; Brown et al., *ReBeL*, NeurIPS 2020
- Perolat et al., *Mastering Stratego with model-free multiagent RL* (DeepNash), Science 2022
- Swisslos Schieber rules: <https://www.swisslos.ch/de/jass/informationen/jass-regeln/schieber-jass.html>